CĐ-06 · Xử lý dữ liệu với pandas

Luyện kiến thức: không tính giờ, chọn xong là biết đúng/sai và xem giải thích. Tiến độ lưu riêng với Luyện đề.

Mã đề **PETER-CD06-5006** · Người làm: Peter: gửi mã này cho mentor khi chữa bài.

Phần A — Nhận biết

Câu 1 · T06-A02 · 3 điểm · Trắc nghiệm · Nhận biết · tình huống

Phần mềm kế toán xuất file `don_hang.csv` theo kiểu châu Âu/Việt Nam: các cột ngăn bằng dấu chấm phẩy, số thập phân dùng dấu phẩy. Nội dung file:

```
ma;gia
D1;1250,5
D2;980,25
```

Lệnh nào đọc được cột `gia` thành số thực `[1250.5, 980.25]`?

- A. `pd.read_csv("don_hang.csv", sep=",", decimal=";")`
- B. `pd.read_csv("don_hang.csv", sep=";", decimal=",")`
- C. `pd.read_csv("don_hang.csv")`
- D. `pd.read_csv("don_hang.csv", sep=";", thousands=",")`

✓ Chính xác!

Đúng! Đọc thêm giải thích chi tiết để nhớ lâu hơn.

Đáp án B. (bản in: A) `sep` chỉ ký tự ngăn cột, `decimal` chỉ ký tự thập phân. File dạng `;` và `,` cần `sep=";", decimal=","`; còn `thousands` dùng cho dấu phân cách hàng nghìn.

- **A:** Đảo vai trò hai tham số: tách cột theo dấu phẩy nên dòng `D1;1250,5` bị cắt sai chỗ và không có cột `gia` nào cả.
- **C:** Mặc định `sep` là dấu phẩy nên cả tiêu đề `ma;gia` thành một cột duy nhất; dữ liệu bị đọc sai cấu trúc.

- **D:** `thousands` coi dấu phẩy là dấu phân cách hàng nghìn rồi bỏ đi, nên 1250,5 thành 12505 và 980,25 thành 98025: sai lệch lớn mà không hề báo lỗi.

▸ Giải thích chi tiết

Thử lại câu này

Câu 2 · T06-A07 · 3 điểm · Trắc nghiệm · Nhận biết · tình huống

Ghép đơn hàng tháng 7 và tháng 8 thành một bảng:

```
t7 = pd.DataFrame({"ma": ["D1", "D2", "D3"]})   # đơn tháng 7
t8 = pd.DataFrame({"ma": ["D4", "D5"]})         # đơn tháng 8
print(pd.concat([t7, t8]).index.tolist())
```

Kết quả in ra là gì?

- **A.** Báo lỗi vì hai bảng có nhãn index trùng nhau
- **B.** `[0, 1, 2, 3, 4]`
- **C.** `[0, 1]`
- **D.** `[0, 1, 2, 0, 1]`

✓ Chính xác!

Tuyệt, sang câu tiếp theo thôi.

Đáp án D. (bản in: A) `pd.concat([a, b])` xếp chồng dòng và giữ nhãn gốc nên index có thể trùng; dùng `ignore_index=True` để đánh lại $0, 1, \ldots, n-1$.

- **A:** `concat` cho phép nhãn trùng; chỉ báo lỗi khi bật `verify_integrity=True`.
- **B:** Chỉ đúng khi truyền `ignore_index=True`; mặc định `concat` giữ nguyên nhãn index của từng bảng.
- **C:** Nhầm `concat` với phép giao các nhãn chung; `concat` theo trục dòng nối toàn bộ dòng của cả hai bảng.

▸ Giải thích chi tiết

Thử lại câu này

Câu 3 · T06-A03 · 3 điểm · Trắc nghiệm · Nhận biết · tình huống

Bảng đặt phòng `bk` có cột `thanh_pho`. Biến Python `ks = "Đà Lạt"` chứa tên thành phố cần lọc. Cách gọi `query` nào trả về đúng các dòng có `thanh_pho` bằng giá trị của biến `ks`?

- **A.** `bk.query("thanh_pho == ks")`
- **B.** `bk.query("thanh_pho == 'ks'")`
- **C.** `bk.query("thanh_pho == @ks")`
- **D.** `bk.query(f"thanh_pho == {ks}")`

✓ Chính xác!

Chuẩn rồi!

Đáp án C. (bản in: D) Trong `DataFrame.query`, tên trần là tên cột, còn `@ten_bien` tham chiếu biến Python ở phạm vi hiện tại, ví dụ `bk.query("thanh_pho == @ks")`.

- **A:** Trong chuỗi `query`, tên không có `@` được hiểu là tên cột; bảng không có cột `ks` nên pandas báo `UndefinedVariableError`.
- **B:** Dấu nháy biến `ks` thành chuỗi hằng 'ks'; không báo lỗi nhưng trả về bảng rỗng vì không có thành phố nào tên là ks.
- **D:** f-string chèn chữ Đà Lạt mà không có dấu nháy, biểu thức thành `thanh_pho == Đà Lạt` sai cú pháp (SyntaxError).

▸ Giải thích chi tiết

Thử lại câu này

Câu 4 · T06-A04 · 3 điểm · Trắc nghiệm · Nhận biết · tình huống

Log web của trang đặt phòng có cột `status` (mã HTTP, kiểu số nguyên). Biểu thức nào lấy đúng các request có mã **404 hoặc 500**?

- **A.** `log.loc[[404, 500]]`

- **B.** `log[log["status"].isin([404, 500])]`
- **C.** `log[log["status"] == [404, 500]]`
- **D.** `log[log["status"] in [404, 500]]`

✓ Chính xác!
Giỏi lắm, giữ nhịp này nhé.

Đáp án B. `Series.isin(values)` trả mặt nạ boolean: True khi phần tử thuộc tập `values`; tương đương `(s == 404) | (s == 500)` nhưng gọn hơn khi tập lớn.

- **A:** `.loc` với list chọn theo **nhãn index** (dòng có nhãn 404, 500), không lọc theo giá trị cột `status`; với index mặc định sẽ báo KeyError hoặc lấy nhầm dòng.
- **C:** So sánh `==` với list là so từng phần tử theo vị trí; độ dài Series khác 2 nên báo `ValueError: Lengths must match`.
- **D:** Toán tử `in` của Python cần một giá trị đúng/sai duy nhất cho cả Series nên báo `ValueError` (truth value is ambiguous).

▸ Giải thích chi tiết

Thử lại câu này

Câu 5 · T06-A08 · 3 điểm · Trắc nghiệm · Nhận biết

Đoạn code sau in ra gì?

```
import numpy as np
import pandas as pd
s = pd.Series([1.5, np.nan, 2.0, np.nan])
print((s == np.nan).sum(), s.isna().sum())
```

- **A.** `0 2`
- **B.** `2 2`
- **C.** `2 0`
- **D.** `0 0`

✓ Chính xác!

Chính xác, bạn nắm chắc phần này.

Đáp án A. (bản in: D) Vì $\mathrm{NaN} \neq \mathrm{NaN}$, phép `s == np.nan` luôn ra False; muốn tìm ô thiếu phải dùng `isna()` / `notna()`.

- **B:** Tưởng `== np.nan` nhận ra ô thiếu; theo chuẩn IEEE 754, NaN không bằng bất kỳ giá trị nào, kể cả chính nó.
- **C:** Đảo ngược vai trò hai biểu thức: chính `isna()` mới là cách phát hiện ô thiếu.
- **D:** `isna()` nhận ra đúng hai ô NaN, nên vế thứ hai là 2.

▸ Giải thích chi tiết

Thử lại câu này

Câu 6 · T06-A06 · 3 điểm · Trắc nghiệm · Nhận biết · tình huống

Log web có cột `path` và `user_id`; request của khách chưa đăng nhập có `user_id` là NaN. Lệnh nào đếm **tổng số request** của mỗi `path`, tính cả request của khách chưa đăng nhập?

- A. `log.groupby("path")["user_id"].count()`
- B. `log.groupby("path")["user_id"].value_counts()`
- C. `log.groupby("path")["user_id"].nunique()`
- D. `log.groupby("path").size()`

✓ Chính xác!
Đúng! Đọc thêm giải thích chi tiết để nhớ lâu hơn.

Đáp án D. (bản in: C) `size()` đếm số dòng của mỗi nhóm, kể cả dòng có NaN; `count()` đếm số giá trị không thiếu của từng cột.

- **A:** `count()` chỉ đếm giá trị **không thiếu**, nên bỏ sót các request có `user_id` là NaN.
- **B:** Đếm theo từng cặp (path, user_id) và mặc định bỏ NaN, nên ra nhiều dòng mỗi path chứ không phải tổng số request.
- **C:** `nunique()` đếm số người dùng **khác nhau** (và cũng bỏ NaN), không phải số request.

▸ Giải thích chi tiết

Thử lại câu này

Câu 7 · T06-A05 · 3 điểm · Trắc nghiệm · Nhận biết · tình huống

Bảng `don` có 8 dòng đơn hàng của 3 khách hàng (cột `ma_kh`, `tien`). Kết quả của

```
t = don.groupby("ma_kh")["tien"].transform("sum")
```

có dạng nào?

- **A.** Series dài 8, chỉ dòng đầu tiên của mỗi khách mang tổng tiền, các dòng còn lại của khách đó là NaN
- **B.** Một số duy nhất là tổng tiền cả bảng, vì transform áp hàm lên toàn bộ cột
- **C.** Series dài 8, cùng index với `don`; mỗi dòng mang tổng tiền của khách sở hữu dòng đó
- **D.** Series dài 3, index là `ma_kh`; mỗi phần tử là tổng tiền của một khách hàng

✓ Chính xác!
Tuyệt, sang câu tiếp theo thôi.

Đáp án C. (bản in: B) `agg` thu mỗi nhóm về 1 dòng (độ dài = số nhóm); `transform` trả kết quả cùng index với bảng gốc (độ dài = số dòng), tiện để gán thành cột mới.

- **A:** `transform` phát (broadcast) giá trị nhóm cho **mọi** dòng của nhóm, không chỉ dòng đầu.
- **B:** Nhầm với `don["tien"].sum()`; sau `groupby`, hàm được áp riêng cho từng nhóm.
- **D:** Đây là kết quả của `groupby(...).sum()` hoặc `agg("sum")`: gộp mỗi nhóm thành một dòng. `transform` thì trả lại đúng hình dạng ban đầu.

▸ Giải thích chi tiết

Thử lại câu này

Câu 8 · T06-A01 · 3 điểm · Trắc nghiệm · Nhận biết · tình huống

Bảng đơn hàng `df` có cột `gia` (nghìn đồng) và `tinh`. Biểu thức nào lọc đúng các đơn có giá **trên 1000** và ở tỉnh **HN**?

A. `df[(df["gia"] > 1000) | (df["tinh"] == "HN")]`

B. `df[df["gia"] > 1000 & df["tinh"] == "HN"]`

C. `df[(df["gia"] > 1000) & (df["tinh"] == "HN")]`

D. `df[df["gia"] > 1000 and df["tinh"] == "HN"]`

✓ Chính xác!

Chuẩn rồi!

Đáp án C. Mặt nạ boolean kết hợp bằng `&` (và), `|` (hoặc), `~` (phủ định) theo từng phần tử; mỗi điều kiện phải nằm trong ngoặc vì `&`, `|` ưu tiên cao hơn phép so sánh: `(a > x) & (b == y)`.

- **A:** `|` là phép HOẶC: kết quả lẫn cả đơn ngoài HN giá trên 1000 lẫn đơn ở HN giá thấp.
- **B:** Thiếu ngoặc: `&` có độ ưu tiên cao hơn `>` và `==`, nên Python tính `1000 & df["tinh"]` trước và báo lỗi kiểu (TypeError).
- **D:** `and` của Python đòi một giá trị đúng/sai duy nhất cho cả Series nên pandas báo `ValueError: The truth value of a Series is ambiguous`; phải dùng `&` để kết hợp theo từng phần tử.

▸ Giải thích chi tiết

Thử lại câu này

Phần B — Thông hiểu

Câu 9 · T06-B09 · 3,5 điểm · Điền kết quả · Thông hiểu

Ngày được nhập theo kiểu Việt Nam (ngày/tháng/năm), có một ngày không tồn tại. Đoạn code sau in ra gì? Ghi đúng một dòng.

```
import pandas as pd
s = pd.Series(["05/03/2026", "31/12/2026", "30/02/2026"])
d = pd.to_datetime(s, format="%d/%m/%Y", errors="coerce")
print(d.isna().sum(), d.min().month, d.max().day)
```

Gõ đúng kết quả in ra, mỗi dòng một dòng…

Kiểm tra

Câu 10 · T06-B03 · 3 điểm · Trắc nghiệm · Thông hiểu · tình huống

Bảng đặt phòng `bk` có các cột theo thứ tự `ma_dat, ngay_dat, so_dem, gia_dem, trang_thai` và index mặc định $0, 1, 2, \ldots$ Biểu thức nào trả về **đúng 3 cột** `so_dem, gia_dem, trang_thai` (giữ mọi dòng)?

- **A.** `bk.loc[:, "so_dem":"trang_thai"]`
- **B.** `bk.iloc[:, 2:4]`
- **C.** `bk.loc["so_dem":"trang_thai"]`
- **D.** `bk.iloc[:, "so_dem":"trang_thai"]`

✓ Chính xác!

Chính xác, bạn nắm chắc phần này.

Đáp án A. (bản in: C) `loc` cắt theo nhãn và **gồm cả** hai đầu mút; `iloc` cắt theo vị trí kiểu Python $[a, b)$. Tương đương: `bk.iloc[:, 2:5]`.

- **B:** `iloc` cắt theo vị trí và **không** lấy đầu mút phải (giống list Python), nên chỉ được 2 cột `so_dem, gia_dem`.
- **C:** Chỉ một bộ chỉ số trong `loc` là cắt theo **nhãn dòng**, không phải cột; kết quả không phải 3 cột cần lấy.
- **D:** `iloc` chỉ nhận số nguyên vị trí; truyền nhãn chuỗi sẽ báo TypeError.

▸ Giải thích chi tiết

Thử lại câu này

Câu 11 · T06-B08 · 3 điểm · Trắc nghiệm · Thông hiểu

Đoạn code sau in ra gì?

```
don = pd.DataFrame({"ma_sp": [1, 2], "gia": [100, 200], "sl": [3, 1]})
sp  = pd.DataFrame({"ma_sp": [1, 2], "gia": [90, 210], "ten": ["Ao", "Mu"]})
r = don.merge(sp, on="ma_sp")
print(r.columns.tolist())
```

A. `['ma_sp', 'gia', 'sl', 'ten']`

B. `['ma_sp', 'gia_x', 'sl', 'gia_y', 'ten']`

C. `['ma_sp', 'gia', 'sl', 'gia', 'ten']`

D. `['ma_sp_x', 'gia_x', 'sl', 'ma_sp_y', 'gia_y', 'ten']`

✓ Chính xác!

Đúng! Đọc thêm giải thích chi tiết để nhớ lâu hơn.

Đáp án B. (bản in: D) Cột không phải khóa mà trùng tên ở hai bảng nhận hậu tố theo `suffixes=("_x", "_y")`; thứ tự cột là: khóa, các cột còn lại của bảng trái, rồi các cột của bảng phải.

- **A:** Tưởng cột trùng tên bị ghi đè; pandas giữ cả hai và thêm hậu tố để phân biệt.
- **C:** pandas không để hai cột cùng tên sau merge; mặc định thêm hậu tố `_x` (bảng trái) và `_y` (bảng phải).
- **D:** Cột khóa trong `on` được gộp làm một, không bị gắn hậu tố.

▸ Giải thích chi tiết

Thử lại câu này

Câu 12 · T06-B06 · 3 điểm · Trắc nghiệm · Thông hiểu · tình huống

Một số đơn hàng bị mất thông tin kênh bán. Đoạn code sau in ra gì và vì sao?

```
don = pd.DataFrame({"kenh": ["web", None, "app", "web", None],
                    "tien": [100, 50, 200, 150, 70]})
print(don["tien"].sum(), don.groupby("kenh")["tien"].sum().sum())
```

A. `570 nan`, vì chỉ cần một khóa NaN là tổng theo nhóm thành NaN

B. `570 570`, vì `groupby` gom mọi khóa NaN vào một nhóm riêng

C. `570 450`, vì `groupby` mặc định bỏ các dòng có khóa NaN (`dropna=True`)

D. `450 450`, vì `sum()` trên cột `tien` cũng bỏ các dòng có `kenh` thiếu

✓ Chính xác!

Tuyệt, sang câu tiếp theo thôi.

Đáp án C. (bản in: B) Tổng toàn cột $= 570$; theo nhóm chỉ còn web 250 và app 200, tổng 450, hụt đúng $50 + 70 = 120$ của hai dòng thiếu kênh. Dùng `dropna=False` để giữ nhóm NaN.

- **A:** NaN ở **khóa** không lan sang giá trị; nhóm NaN đơn giản là bị bỏ, các nhóm còn lại vẫn có tổng bình thường.
- **B:** Chỉ đúng khi gọi `groupby("kenh", dropna=False)`; mặc định các dòng khóa thiếu bị loại khỏi kết quả.
- **D:** `don["tien"].sum()` không liên quan cột `kenh`; cả 5 giá trị tiền đều được cộng, ra 570.

▸ Giải thích chi tiết

Thử lại câu này

Câu 13 · T06-B07 · 3 điểm · Trắc nghiệm · Thông hiểu · tình huống

Bảng `don` (cột `ma_don`, `ma_kh`) và bảng danh mục khách `kh` (khóa `ma_kh` không trùng). Đội kiểm toán dữ liệu cần lấy **các đơn có `ma_kh` không tồn tại trong `kh`** (đơn mồ côi). Biểu thức nào đúng?

- **A.** `don[~don["ma_kh"].isin(kh["ma_kh"])]`
- **B.** `don.merge(kh, on="ma_kh", how="inner")`
- **C.** `don.merge(kh, on="ma_kh", how="left", indicator=True).query("_merge == 'right_only'")`
- **D.** `don[don["ma_kh"].isin(kh["ma_kh"])]`

✓ Chính xác!
Chuẩn rồi!

Đáp án A. (bản in: C) Anti-join: giữ dòng bên trái không có khóa bên phải, viết `don[~don["ma_kh"].isin(kh["ma_kh"])]` hoặc left merge với `indicator=True` rồi lọc `_merge == "left_only"`.

- **B:** Inner join chỉ giữ các đơn khớp được khách, nên đơn mồ côi bị loại hết.
- **C:** Với left join, cột `_merge` chỉ có `both` hoặc `left_only`; đơn mồ côi mang nhãn `left_only`, lọc `right_only` luôn ra bảng rỗng.

- **D:** Thiếu `~` : đây là các đơn **có** khách hợp lệ, ngược với yêu cầu.

▸ Giải thích chi tiết

Thử lại câu này

Câu 14 · T06-B02 · 3 điểm · Trắc nghiệm · Thông hiểu

Theo **pandas ≥ 3.0** (Copy-on-Write là mặc định), đoạn code sau in ra gì?

```
df = pd.DataFrame({"phong": ["101", "102", "103"],
                   "gia": [800, 950, 1200]})
df[df["gia"] > 900]["gia"] = 1000
print(df["gia"].tolist())
```

- **A.** `[800, 1000, 1000]` ; phép gán ghi xuyên qua mặt nạ vào thẳng `df`
- **B.** Chương trình dừng với một exception vì pandas 3.0 cấm hẳn cú pháp gán chuỗi
- **C.** `[800, 950, 1200]` ; `df` giữ nguyên vì phép gán rơi vào một bản sao tạm
- **D.** `[1000, 1000, 1000]` ; điều kiện bị bỏ qua và cả cột được gán

✓ Chính xác!

Giỏi lắm, giữ nhịp này nhé.

Đáp án C. (bản in: A) Với Copy-on-Write, `df[mask]["gia"] = 1000` không bao giờ sửa `df` . Cách đúng là gán một bước: `df.loc[df["gia"] > 900, "gia"] = 1000` .

- **A:** Đây là kỳ vọng sai về gán chuỗi (chained assignment): `df[mask]` tạo đối tượng mới, gán vào đó không chạm tới `df` .
- **B:** pandas 3.0 chỉ phát cảnh báo `ChainedAssignmentError` (một warning), chương trình vẫn chạy tiếp và in danh sách cũ.
- **D:** Mặt nạ vẫn được áp dụng cho đối tượng tạm; vấn đề là kết quả gán không quay về `df` , chứ không phải điều kiện bị bỏ qua.

▸ Giải thích chi tiết

Thử lại câu này

Câu 15 · T06-B04 · 3,5 điểm · Điền kết quả · Thông hiểu · tình huống

Thống kê log web theo đường dẫn. Đoạn code sau in ra gì? Ghi đúng 2 dòng.

```
import pandas as pd
log = pd.DataFrame({
    "path": ["/home", "/rooms", "/home", "/rooms", "/home", "/book"],
    "ip":   ["a", "b", "a", "a", "c", "b"],
})
g = log.groupby("path")["ip"]
print(g.size().to_dict())
print(g.nunique().to_dict())
```

Gõ đúng kết quả in ra, mỗi dòng một dòng…

Kiểm tra

Câu 16 · T06-B05 · 3 điểm · Trắc nghiệm · Thông hiểu

Cho bảng doanh thu theo ngày và kênh:

```
df = pd.DataFrame({"ngay": ["T2", "T2", "T3"],
                   "kenh": ["web", "web", "app"],
                   "tien": [100, 300, 200]})
p1 = df.pivot(index="ngay", columns="kenh", values="tien")
p2 = df.pivot_table(index="ngay", columns="kenh", values="tien")
```

Điều gì xảy ra?

- **A.** Cả hai đều báo ValueError vì cặp (T2, web) bị trùng
- **B.** Cả hai đều chạy được, ô (T2, web) cùng bằng 400 vì mặc định cộng dồn các dòng trùng
- **C.** `p1` giữ dòng xuất hiện sau nên ô (T2, web) bằng 300; `p2` cho 400
- **D.** `p1` báo ValueError vì cặp (T2, web) xuất hiện hai lần; `p2` chạy được và ô (T2, web) bằng 200

✓ Chính xác!

Đúng! Đọc thêm giải thích chi tiết để nhớ lâu hơn.

Đáp án D. `pivot` chỉ định hình lại, đòi mỗi cặp (index, columns) là duy nhất; `pivot_table` gộp trùng bằng `aggfunc` (mặc định trung bình): $(100 + 300)/2 = 200$.

- **A:** `pivot_table` được thiết kế để gộp các cặp trùng bằng `aggfunc`, nên không báo lỗi.
- **B:** `pivot` không gộp gì cả, còn `pivot_table` mặc định `aggfunc="mean"` chứ không phải `sum`.
- **C:** `pivot` không tự chọn dòng nào khi trùng mà báo lỗi; `pivot_table` lấy trung bình, không lấy tổng.

▸ Giải thích chi tiết

Thử lại câu này

Câu 17 · T06-B01 · 3,5 điểm · Điền kết quả · Thông hiểu · tình huống

Log web của một trang đặt phòng ghi thời gian phản hồi `ms`. Đoạn code sau in ra gì? Ghi đúng một dòng.

```
import pandas as pd
log = pd.DataFrame({
    "path": ["/api/rooms", "/home", "/api/book", "/home", "/api/rooms", "/about"],
    "ms":   [120, 300, 450, 80, 300, 100],
})
a = log["ms"].between(100, 300).sum()
b = log.loc[~log["path"].str.startswith("/api"), "ms"].sum()
print(a, b)
```

Gõ đúng kết quả in ra, mỗi dòng một dòng…

Kiểm tra

Câu 18 · T06-B10 · 3 điểm · Trắc nghiệm · Thông hiểu · tình huống

File đặt phòng có cột `so_khach` bị thiếu ở vài dòng nên pandas đọc thành float64 (ví dụ `2.0, NaN, 3.0`). Kỹ sư muốn chuyển cột về số nguyên mà vẫn giữ thông tin "thiếu". Nhận định nào đúng?

- **A.** `astype(int)` chạy được, cột thành int64 và các ô thiếu vẫn là NaN

- **B.** pandas không thể lưu cột số nguyên có ô thiếu, nên bắt buộc `dropna()` trước khi đổi kiểu
- **C.** `astype(int)` báo lỗi vì NaN không có dạng int64; `astype("Int64")` cho số nguyên với ô thiếu là `<NA>`
- **D.** `astype(int)` chạy được và tự thay NaN bằng 0, không cần xử lý gì thêm

✓ **Chính xác!**

Chuẩn rồi!

Đáp án C. (bản in: A) Kiểu `int64` không biểu diễn được NaN; kiểu nullable `"Int64"` (chữ I hoa) dùng mặt nạ riêng nên giữ được ô thiếu: `[2, <NA>, 3]`.

- **A:** NaN là giá trị số thực; kiểu int64 của NumPy không có chỗ cho NaN nên không thể vừa là int64 vừa giữ NaN.
- **B:** pandas có kiểu nguyên cho phép thiếu (nullable) `Int64` với giá trị thiếu `pd.NA`, không cần bỏ dòng.
- **D:** pandas không âm thầm biến NaN thành 0 (vì như vậy làm sai số liệu); nó báo `IntCastingNaNError`.

▸ Giải thích chi tiết

Thử lại câu này

Phần C — Vận dụng

Câu 19 · T06-C03 · 3 điểm · Trắc nghiệm · Vận dụng · tình huống

Bảng đặt phòng của hai khách sạn:

```
bk = pd.DataFrame({"ks":  ["HL", "DL", "HL", "DL", "HL"],
                   "ma":  ["B1", "B2", "B3", "B4", "B5"],
                   "gia": [900, 1200, 1500, 800, 1100]})
```

Biểu thức nào trả về, với mỗi khách sạn, **nguyên dòng** đặt phòng có giá cao nhất (đúng cặp mã và giá)?

- **A.** `bk.groupby("ks").max()`
- **B.** `bk.groupby("ks")["gia"].max()`

C. `bk.sort_values("gia").groupby("ks").head(1)`

D. `bk.loc[bk.groupby("ks")["gia"].idxmax()]`

✓ Chính xác!

Giỏi lắm, giữ nhịp này nhé.

Đáp án D. (bản in: B) `groupby(...)["gia"].idxmax()` trả nhãn dòng có giá lớn nhất trong mỗi nhóm (DL: 1, HL: 2); `bk.loc[...]` lấy nguyên các dòng đó: B2 và B3.

- **A:** `max()` lấy giá trị lớn nhất của **từng cột độc lập**: với HL ra `ma = B5` (chuỗi lớn nhất) và `gia = 1500`, ghép sai cặp vì B5 có giá 1100.
- **B:** Chỉ trả về Series giá lớn nhất, mất cột `ma` nên không biết booking nào.
- **C:** Sắp tăng dần rồi lấy dòng đầu nên được booking **rẻ nhất** (B1, B4); cần `ascending=False`.

▸ Giải thích chi tiết

Thử lại câu này

Câu 20 · T06-C06 · 3,5 điểm · Điền kết quả · Vận dụng · tình huống

Phân tích giờ truy cập trong log web (chú ý request cuối rơi sang ngày hôm sau). Đoạn code sau in ra gì? Ghi đúng 2 dòng.

```
import pandas as pd
log = pd.DataFrame({"ts": pd.to_datetime([
    "2026-10-12 08:05:00", "2026-10-12 08:59:30", "2026-10-12 09:00:00",
    "2026-10-12 23:10:00", "2026-10-13 00:20:00"])})
print(log["ts"].dt.hour.value_counts().sort_index().to_dict())
print((log["ts"].max() - log["ts"].min()).total_seconds() / 3600)
```

Gõ đúng kết quả in ra, mỗi dòng một dòng…

Kiểm tra

Câu 21 · T06-C02 · 3,5 điểm · Điền kết quả · Vận dụng · tình huống

Tính tỷ trọng mỗi đơn trong tổng chi tiêu của chính khách hàng đó. Đoạn code sau in ra gì? Ghi đúng 2 dòng.

```
import pandas as pd
don = pd.DataFrame({
    "kh":   ["An", "Binh", "An", "Chi", "Binh", "An"],
    "tien": [200, 300, 600, 500, 200, 200],
})
don["ty_le"] = don["tien"] / don.groupby("kh")["tien"].transform("sum")
print(don["ty_le"].tolist())
print(don.loc[don["ty_le"] >= 0.5, "kh"].tolist())
```

Gõ đúng kết quả in ra, mỗi dòng một dòng…

Kiểm tra

Câu 22 · T06-C08 · 3 điểm · Trắc nghiệm · Vận dụng · tình huống

Giá phòng bị thiếu được điền bằng giá trung bình của **cùng loại phòng**. Đoạn code sau in ra gì?

```
bk = pd.DataFrame({"loai": ["Std", "Std", "Deluxe", "Std", "Deluxe", "Deluxe"],
                   "gia":  [600, None, 1500, 900, None, 1300]})
bk["gia2"] = bk["gia"].fillna(bk.groupby("loai")["gia"].transform("mean"))
print(bk["gia2"].tolist())
```

- ○ **A.** `[600.0, 750.0, 1500.0, 900.0, 1400.0, 1300.0]`
- ○ **B.** `[600.0, 1075.0, 1500.0, 900.0, 1075.0, 1300.0]`
- ○ **C.** `[600.0, nan, 1500.0, 900.0, nan, 1300.0]`
- ○ **D.** `[600.0, 600.0, 1500.0, 900.0, 900.0, 1300.0]`

Câu 23 · T06-C04 · 3,5 điểm · Điền kết quả · Vận dụng · tình huống

Gộp đơn hàng từ kênh web và kênh app (hai bảng có cột khác nhau). Đoạn code sau in ra gì? Ghi đúng một dòng.

```
import pandas as pd
web = pd.DataFrame({"ma": ["W1", "W2", "W3"], "tien": [100, 250, 80],
                    "coupon": ["GIAM10", None, "FREESHIP"]})
app = pd.DataFrame({"ma": ["A1", "A2"], "tien": [300, 120],
```

```
                    "thiet_bi": ["ios", "android"]})
tat_ca = pd.concat([web, app], ignore_index=True)
print(tat_ca.shape, tat_ca["coupon"].isna().sum(), tat_ca["thiet_bi"].notna().sum())
```

Gõ đúng kết quả in ra, mỗi dòng một dòng…

Kiểm tra

Câu 24 · T06-C01 · 3,5 điểm · Điền kết quả · Vận dụng · tình huống

Bảng đặt phòng của một khách sạn (`dem` : số đêm, `gia` : nghìn đồng/đêm). Đoạn code sau in ra gì? Ghi đúng một dòng.

```
import pandas as pd
bk = pd.DataFrame({
    "ma":   ["B1", "B2", "B3", "B4", "B5", "B6"],
    "loai": ["Std", "Deluxe", "Std", "Suite", "Deluxe", "Std"],
    "dem":  [2, 3, 5, 1, 3, 4],
    "gia":  [700, 1500, 650, 3000, 1400, 700],
})
sub = bk.query("loai != 'Std' or dem >= 4")
top = sub.sort_values(["dem", "gia"], ascending=[False, True])
print(top["ma"].head(3).tolist(), top["gia"].iloc[-1])
```

Gõ đúng kết quả in ra, mỗi dòng một dòng…

Kiểm tra

Câu 25 · T06-C05 · 3 điểm · Trắc nghiệm · Vận dụng · tình huống

Bảng `don` lấy từ cơ sở dữ liệu có `ma_kh` kiểu số nguyên; bảng hạng thành viên `kh` đọc từ file JSON nên `ma_kh` là chuỗi:

```
don = pd.DataFrame({"ma_kh": [101, 102, 103], "tien": [500, 200, 350]})
kh  = pd.DataFrame({"ma_kh": ["101", "102", "104"],
                    "hang": ["Vang", "Bac", "Dong"]})
r = don.merge(kh, on="ma_kh", how="left")
```

Điều gì xảy ra ở dòng `merge` và cách xử lý hợp lý là gì?

- **A.** Báo KeyError vì cột `ma_kh` phải được đặt làm index trước khi merge
- **B.** Chạy được: pandas tự ép "101" thành 101 nên đơn của khách 101 và 102 có hạng
- **C.** Chạy được nhưng cột `hang` toàn NaN vì 101 khác "101"; đổi sang `how="inner"` là xong
- **D.** Báo ValueError vì hai cột khóa khác kiểu (số nguyên và chuỗi); cần ép cùng kiểu, ví dụ `kh["ma_kh"].astype(int)`, rồi mới merge

✓ Chính xác!

Chính xác, bạn nắm chắc phần này.

Đáp án D. (bản in: C) pandas từ chối merge khóa int64 với khóa chuỗi (`ValueError: You are trying to merge on int64 and ... columns`). Chuẩn hóa kiểu khóa trước: sau khi ép, khách 101, 102 có hạng, 103 là NaN.

- **A:** `merge(on=...)` làm việc trực tiếp trên cột, không cần đặt index; lỗi thật nằm ở kiểu dữ liệu của khóa.
- **B:** pandas không tự đổi kiểu khóa khi merge; ép ngầm như vậy dễ che lỗi dữ liệu nên nó từ chối.
- **C:** Với khóa số nguyên và chuỗi pandas báo lỗi ngay; hơn nữa đổi sang inner join cũng không làm 101 khớp với "101".

▸ Giải thích chi tiết

Thử lại câu này

Câu 26 · T06-C09 · 6 điểm · Code viết tay · Vận dụng · tình huống

Log web của trang đặt phòng khách sạn sau khi tách trường được đưa vào DataFrame `log` với các cột:

- `ts`: chuỗi thời điểm dạng `"YYYY-MM-DD HH:MM:SS"`; một số dòng hỏng (ví dụ `"N/A"` hoặc chuỗi rỗng).
- `status`: mã HTTP (số nguyên).
- `ms`: thời gian phản hồi ở dạng **chuỗi**; request bị timeout ghi `"-"`.

Ví dụ:

```
log = pd.DataFrame({
    "ts": ["2026-10-12 08:10:00", "2026-10-12 08:40:00", "N/A",
           "2026-10-12 10:05:00", ""],
    "status": [200, 503, 200, 200, 500],
```

```
    "ms": ["120", "-", "90", "300", "50"],
})
```

Viết (trên giấy) hàm `bao_cao_gio(log)` bằng pandas, trả về DataFrame có **index là mốc đầu mỗi giờ**, liên tục từ giờ của bản ghi hợp lệ sớm nhất đến giờ của bản ghi hợp lệ muộn nhất (**kể cả giờ không có request**), gồm các cột:

Cột	Ý nghĩa
`so_req`	số request trong giờ (số nguyên; giờ trống là 0)
`ty_le_loi`	tỷ lệ request có `status >= 500` (giờ trống là 0.0)
`ms_tb`	trung bình `ms` của các request có `ms` hợp lệ (không có thì NaN)

Yêu cầu: bỏ các dòng có `ts` không chuyển được thành thời điểm; không làm thay đổi `log` đầu vào. Với ví dụ trên, kết quả có 3 dòng 08:00, 09:00, 10:00 với `so_req` = 2, 0, 1.

```
Viết code Python như trên giấy (không chạy được)…
```

Xem bài mẫu & rubric Viết thử trước rồi hãy mở.

Câu 27 · T06-C07 · 3,5 điểm · Điền kết quả · Vận dụng · tình huống

Tính số đêm lưu trú từ ngày nhận phòng `ci` và ngày trả phòng `co`. Đoạn code sau in ra gì? Ghi đúng 2 dòng.

```
import pandas as pd
bk = pd.DataFrame({
    "ci": pd.to_datetime(["2026-07-30", "2026-08-02", "2026-07-05", "2026-08-10"]),
    "co": pd.to_datetime(["2026-08-02", "2026-08-03", "2026-07-08", "2026-08-14"]),
})
bk["dem"] = (bk["co"] - bk["ci"]).dt.days
print(bk["dem"].tolist())
print(bk.groupby(bk["ci"].dt.month)["dem"].sum().to_dict())
```

```
Gõ đúng kết quả in ra, mỗi dòng một dòng…
```

Kiểm tra

Phần D — Vận dụng cao

Câu 28 · T06-D01 · 3 điểm · Trắc nghiệm · Vận dụng cao · tình huống

Chuỗi khách sạn có 3 cơ sở. Bảng `bk` gồm 20 booking (cột `ks`, cột `huy` = 1 nếu booking bị hủy) được tóm tắt thành:

```
r = bk.groupby("ks")["huy"].agg(n="size", rate="mean")
```

ks	n	rate
DL	8	0.25
HL	2	0.50
SG	10	0.10

Ban giám đốc hỏi **tỷ lệ hủy của toàn chuỗi** (số booking bị hủy chia tổng số booking). Chỉ dùng `r`, biểu thức nào cho đúng con số đó, và giá trị bằng bao nhiêu?

- **A.** `r["rate"].mean()`, bằng khoảng 0,283
- **B.** `(r["rate"] * r["n"]).sum() / len(r)`, bằng khoảng 1,333
- **C.** `(r["rate"] * r["n"]).sum() / r["n"].sum()`, bằng 0,2
- **D.** `r["rate"].median()`, bằng 0,25

✓ Chính xác!

Chuẩn rồi!

Đáp án C. (bản in: A) Tỷ lệ chung là trung bình có trọng số: $\frac{\sum_i n_i \cdot rate_i}{\sum_i n_i} = \frac{2+1+1}{20} = 0{,}2$, bằng đúng `bk["huy"].mean()`.

- **A:** Trung bình của các tỷ lệ coi HL (2 booking) nặng ngang SG (10 booking); đây là trung bình không trọng số, không phải tỷ lệ toàn chuỗi.
- **B:** Tử số đúng (4 booking bị hủy) nhưng chia cho số khách sạn thay vì tổng số booking; tỷ lệ lớn hơn 1 là dấu hiệu sai.
- **D:** Trung vị các tỷ lệ cũng bỏ qua quy mô từng cơ sở; chỉ trùng tỷ lệ chung khi mọi cơ sở có cùng số booking.

▸ Giải thích chi tiết

Thử lại câu này

Câu 29 · T06-D03 · 6 điểm · Code viết tay · Vận dụng cao · tình huống

Một chuỗi khách sạn lưu hai bảng:

- `bk` (booking): `ma_dat`, `ma_ks`, `checkin`, `checkout` (datetime), `gia_dem` (nghìn đồng/đêm), `trang_thai` (`"ok"` hoặc `"huy"`).
- `ks` (danh mục khách sạn, khóa `ma_ks` không trùng): `ma_ks`, `ten`, `so_phong`.

Ví dụ:

```
bk = pd.DataFrame({
    "ma_dat": ["D1", "D2", "D3", "D4", "D5"],
    "ma_ks":  ["HL", "HL", "DL", "DL", "HL"],
    "checkin":  pd.to_datetime(["2026-07-01", "2026-07-03", "2026-07-02", "2026-07-05", "20
    "checkout": pd.to_datetime(["2026-07-03", "2026-07-04", "2026-07-05", "2026-07-06", "20
    "gia_dem": [1000, 1200, 800, 900, 1100],
    "trang_thai": ["ok", "huy", "ok", "ok", "ok"],
})
ks = pd.DataFrame({"ma_ks": ["DL", "HL", "SG"],
                   "ten": ["Đà Lạt", "Hạ Long", "Sài Gòn"],
                   "so_phong": [20, 30, 50]})
```

Viết (trên giấy) hàm `bao_cao_ks(bk, ks)` trả về DataFrame **một dòng cho mỗi khách sạn trong `ks`** với các cột theo thứ tự `ma_ks, ten, so_dat, ty_le_huy, dem_ban, doanh_thu`:

Cột	Ý nghĩa
`so_dat`	tổng số booking (kể cả bị hủy)
`ty_le_huy`	số booking hủy chia `so_dat`
`dem_ban`	tổng số đêm của các booking **không hủy** (số đêm = `checkout - checkin` tính theo ngày)
`doanh_thu`	tổng `số đêm × gia_dem` của các booking **không hủy**

Yêu cầu: khách sạn không có booking nào vẫn có dòng với `so_dat`, `dem_ban`, `doanh_thu` bằng 0 (số nguyên) và `ty_le_huy` bằng 0.0; sắp `doanh_thu` giảm dần, bằng nhau thì `ma_ks` tăng dần; index đánh lại từ 0; không làm thay đổi `bk`, `ks`. Với ví dụ, thứ tự dòng là HL (doanh thu 4200), DL (3300), SG (0).

Viết code Python như trên giấy (không chạy được)…

Xem bài mẫu & rubric Viết thử trước rồi hãy mở.

Câu 30 · T06-D02 · 3 điểm · Trắc nghiệm · Vận dụng cao · tình huống

Giá phòng của một khách sạn thay đổi theo mùa; bảng `gia` ghi giá có hiệu lực **từ** ngày `tu_ngay`. Gán giá cho từng booking theo ngày nhận phòng:

```
gia = pd.DataFrame({"tu_ngay": pd.to_datetime(["2026-06-01", "2026-07-01", "2026-07-15"]),
                    "gia_dem": [900, 1200, 1500]})
bk = pd.DataFrame({"ma": ["B1", "B2", "B3"],
                   "ngay": pd.to_datetime(["2026-06-20", "2026-07-15", "2026-07-10"])})
r = pd.merge_asof(bk.sort_values("ngay"), gia, left_on="ngay", right_on="tu_ngay")
print(r["ma"].tolist(), r["gia_dem"].tolist())
```

Kết quả in ra là gì?

- A. `['B1', 'B3', 'B2'] [1200, 1500, 1500]`
- B. `['B1', 'B2', 'B3'] [900, 1500, 1200]`
- C. `['B1', 'B3', 'B2'] [900, 1200, 1200]`
- D. `['B1', 'B3', 'B2'] [900, 1200, 1500]`

✗ Chưa đúng: bạn chọn B, đáp án đúng là D.

Đọc lý do bên dưới rồi bấm "Thử lại câu này".

Đáp án D. (bản in: B) `merge_asof` (mặc định `direction="backward"`) ghép mỗi dòng trái với dòng phải có khóa lớn nhất ≤ khóa trái: 20/6 nhận 900, 10/7 nhận 1200, 15/7 nhận 1500. Cả hai bảng phải được sắp theo khóa.

- **A:** Đây là hướng `direction="forward"` (lấy mốc gần nhất phía sau); mặc định là `"backward"`, lấy mốc gần nhất **không sau** ngày booking.
- **B:** Ghép giá thì đúng nhưng giữ thứ tự gốc của `bk`; `merge_asof` trả kết quả theo thứ tự của bảng trái **đã sắp** theo `ngay`.

- **C:** Tưởng mốc trùng đúng ngày không được tính; mặc định `allow_exact_matches=True` nên B2 (15/7) nhận giá 1500 có hiệu lực từ 15/7.

▸ **Giải thích chi tiết**

Thử lại câu này